

Product Requirement Document (PRD)

Project: EagleTech Store

Version: 1.1 (Brevo Integrated)

Platform: Web App & Mobile Ready

1. EXECUTIVE SUMMARY & CORE OBJECTIVES

Objective: Build a high-performance e-commerce store and field service booking platform. The system handles hardware sales (laptops, phones, storage drives, enclosures, accessories) and Starlink kit orders paired with professional installation and network distribution services.

1.1 Key Architectural Principles

- Decoupled Mobile-Ready Back-End:** Built with Next.js 14 App Router and Supabase, providing shared REST/GraphQL endpoints for seamless future iOS & Android app expansion.
- Database & Security:** PostgreSQL via Supabase with Row-Level Security (RLS) policies for complete multi-tenant/customer data protection.
- Authentication:** Google OAuth 2.0 via Google Cloud Console integrated into Supabase Auth.
- Payment Gateways:** Payoutstack API for credit/debit cards, bank transfers, and USSD.
- Transactional Emails:** Brevo API (via @getbrevo/brevo SDK) handling up to 300 free automated order confirmation receipts daily.

2. TECH STACK SPECIFICATION

Layer	Technology	Role & Selection Rationale
Frontend	Next.js 14+ / React	Server-Side Rendering (SSR) for product SEO & unified API route support.
UI Framework	Tailwind CSS + Shadcn UI	Clean, responsive design system with dark/light mode support.
Database Engine	Supabase / PostgreSQL	Relational DB with built-in Auth, instant APIs, and storage bucket management.
Authentication	Supabase Auth	Google Cloud OAuth 2.0 provider flow for instant 1-click user onboarding.
Payments	Payoutstack Webhooks	Automated transaction verification via cryptographic HMAC signatures.
Email Service	Brevo Transactional API	Reliable delivery of HTML order receipts (300 free daily emails).

3. CATALOG STRUCTURE & TAXONOMY

 Laptops Brands: Apple (MacBook Air/Pro), Dell (XPS, Latitude), HP (Spectre, Envy), Asus (ROG, ZenBook). Attributes: RAM, CPU, Storage size, Condition, Year specifically for macbooks.	 Smartphones Brands: Apple (iPhones), Samsung, Redmi/Xiaomi, Google Pixel, Tecno. Attributes: Internal Storage, Color, Battery Health, Grade.
--	--

4. CHECKOUT & INTEGRATION ARCHITECTURE

- **Product Selection:** Multi-filtering by brand, size, price, and optional service add-ons.
- **Starlink Service Options:** Users can purchase Starlink kits solo OR select an installation package (Roof mount, cabling, Wi-Fi access point distribution) at checkout.
- **Google OAuth:** One-click authentication generating user profile records in Supabase automatically via trigger functions.
- **Order Execution:** Next.js initializes a Paystack transaction -> User completes payment -> Paystack sends HMAC-signed webhook -> Next.js verifies payload, updates order status to SUCCESSFUL, and invokes Brevo API.

5. DATABASE SCHEMA (POSTGRESQL / SUPABASE)

```
-- 1. USER PROFILES TABLE
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  full_name TEXT,
  phone_number TEXT,
  address TEXT,
  role TEXT DEFAULT 'customer',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Trigger to auto-create profile on Google OAuth Sign-in
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, full_name)
  VALUES (new.id, new.raw_user_meta_data->'full_name');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

-- 2. PRODUCTS & VARIANTS TABLE
CREATE TABLE public.products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  description TEXT,
  price DECIMAL(12, 2) NOT NULL,
  stock_quantity INT DEFAULT 0,
  images TEXT[] DEFAULT '{}',
  is_service BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. ORDERS TABLE
CREATE TABLE public.orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.profiles(id),
  total_amount DECIMAL(12, 2) NOT NULL,
  payment_status TEXT DEFAULT 'PENDING',
  order_status TEXT DEFAULT 'PROCESSING',
  paystack_reference TEXT UNIQUE,
  shipping_address TEXT NOT NULL,
  installation_notes TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

6. API INTEGRATION CODE SPECIFICATIONS

6.1 Brevo Transactional Mailer Helper (lib/brevo.ts)

```
import { BrevoClient } from '@getbrevo/brevo';

const brevo = new BrevoClient({ apiKey: process.env.BREVO_API_KEY || '' });

export async function sendOrderConfirmationEmail(data: {
  email: string;
  customerName: string;
  orderId: string;
  totalAmount: number;
  items: Array<{ name: string; quantity: number; price: number }>;
}) {
  const itemRows = data.items
    .map(i => `<li>${i.name} (x${i.quantity}) - ₹${i.price.toLocaleString()}</li>`)
    .join('');

  const htmlContent = `
    <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px; border:
1px solid #e5e7eb; border-radius: 8px;">
      <h2 style="color: #111827;">Payment Confirmed! 🎉</h2>
      <p>Hello <strong>${data.customerName}</strong>,</p>
      <p>We have received your payment for order <strong>#${data.orderId}</strong>.</p>
      <h3>Items Summary:</h3>
      <ul>${itemRows}</ul>
      <p><strong>Total Paid:</strong> ₹${data.totalAmount.toLocaleString()}</p>
      <p style="background: #f3f4f6; padding: 10px; border-radius: 4px;">
        💡 If you ordered a Starlink setup, our engineers will call you within 24 hours.
      </p>
    </div>`;

  return await brevo.transactionalEmails.sendTransacEmail({
    subject: `Order Confirmation #${data.orderId}`,
    htmlContent: htmlContent,
    sender: { name: process.env.SENDER_NAME || 'TechHub Store', email: process.env.SENDER_EMAIL! },
    to: [{ email: data.email, name: data.customerName }],
  });
}
```

6.2 Paystack Webhook Handler (app/api/webhooks/paystack/route.ts)

```
import { NextResponse } from 'next/server';
import crypto from 'crypto';
import { supabaseAdmin } from '@lib/supabaseAdmin';
import { sendOrderConfirmationEmail } from '@lib/brevo';

export async function POST(req: Request) {
  const rawBody = await req.text();
  const signature = req.headers.get('x-paystack-signature');

  const hash = crypto.createHmac('sha512', process.env.PAYSTACK_SECRET_KEY!)
    .update(rawBody).digest('hex');

  if (hash !== signature) return NextResponse.json({ message: 'Invalid signature' }, { status: 400 });

  const event = JSON.parse(rawBody);

  if (event.event === 'charge.success') {
    const { reference, customer } = event.data;

    const { data: order } = await supabaseAdmin
      .from('orders')
      .update({ payment_status: 'SUCCESSFUL' })
      .eq('paystack_reference', reference)
      .select('*', profiles(full_name), order_items(*, products(title)))
      .single();
  }
}
```